ChatGPT

vc consegue acessar um link que eu te passar e avaliar o site, tela a tela?

Consigo sim — **manda o link aí** 👍

Só alinhando expectativa rapidinho (sem enrolação estilo daily):

- 🔎 Consigo **analisar o site inteiro navegando pelas páginas principais**
- 🧠 Avalio **UX/UI, performance percebida, copy, acessibilidade, fluxo de navegação**
- 🐛 Aponto **bugs, inconsistências e possíveis melhorias**
- 💡 Posso sugerir **melhorias práticas (nível "isso aqui daria ticket no backlog")**

⚠️ Único detalhe:
Se tiver páginas que exigem login, acesso interno ou algo assim, você vai precisar me mandar prints ou descrever — aí eu entro no modo "QA + UX senior chato" sem dó 😄

Manda o link e já começo o review 🚀

tenho esse portfolio meu que preciso fazer um remake dele:
https://victor-hanszman-portfolio.vercel.app/
quero refazer tudo e inclusive estou começando o projeto do zero, ate em outra tecnologia... o código fonte do antigo no git é esse:
https://github.com/Hanszman/victor_hanszman_portfolio-old (em react com js, sem ts) e o do novo vai ser esse https://github.com/Hanszman/hans-portfolio-app (com angular)... e pretendo agora utilizar uma lib de design-system que desenvolvi recentemente para reutilizar os componentes, esse é o link dela:
https://hans-ui-design-lib-doc.vercel.app/?path=/docs/getting-started--docs
e esse é o repositório dela:
https://github.com/Hanszman/hans-ui-design-lib fiz a lib em react + ts + vite e publiquei via npm (pra react) e cdn/module-federation (pra angular)...

repare também que o portfolio antigo não tinha um backend, tem somente o frontend que utiliza os dados via arquivos json hardcode que tenho no sistema antigo... e dessa vez quero fazer um backend mas ainda estou tentando decidir entre .net e nest.js... mas enfim! preciso que vc me dê dicas de como fazer esse remake/refatoração... e preciso que vc analise as telas do antigo, verifique todas elas (home, experiences, skills, projects), alem de cada item nelas (card de tecnologia, graficos, informações, etc) e dê sua opinião de como posso melhorar, tanto em disponbilização de informação pelas telas, como sobre o que mostrar, como sobre o proprio design das telas, como ficar mais bonito e moderno, etc... minha intenção é mostrar todos os projetos nos quais ja trabalhei/estudei, alem de todas as tecnologias que sei, ja mexi, ja estudei ou estou estudando... minhas faculdades, etc... e quero colocar o link do git e do projeto publicado (deployado) de cada um dos mais importantes (highlights)... quero ter essa sessão de highlights (meus projetos principais) bem visível no site, muito provavelmente na tela inicial... mas enfim! me ajude dando dicas de como posso melhorar

Pensou por 2m 40s >

Sim — e já fui direto no osso.

Diagnóstico honesto: teu portfólio antigo tem **muito conteúdo bom**, mas hoje ele passa mais vibe de **inventário técnico** do que de **vitrine estratégica**. Dá pra ver que você quis mostrar profundidade, variedade e histórico, só que a informação está muito espalhada e com pouca hierarquia visual. Inclusive, no código antigo já existem vários TODOs que apontam exatamente isso: melhorar cards, responsividade, links de prod/git, screenshots, modais, paginação, highlights, gráficos e organização das skills. GitHub +1

Também confirmei que o app antigo é dividido nas rotas **Home, Experiences, Skills e Projects**, e o novo app já nasceu em **Angular + TypeScript**, com Tailwind/SCSS, testes e arquitetura standalone/routing. A tua design system também já está preparada para uso em **React via npm** e em **Angular via CDN/web components**, com `CUSTOM_ELEMENTS_SCHEMA` quando necessário. GitHub +2

Um detalhe: o site publicado atual não foi plenamente renderizado pela ferramenta porque ele exige JavaScript no client, então a minha análise visual “tela a tela” ficou

baseada no **código-fonte, estrutura das páginas e dados JSON**, não num render visual completo do deploy. victor-hanszman... +4

Minha leitura geral do remake

O remake tem que sair de:

"olha tudo que eu já mexi"

para:

"esse é o tipo de dev que eu sou, esse é o meu nível, esses são meus melhores cases, e aqui está a prova."

Essa virada muda tudo.

Em vez de o foco principal ser "quantidade de tech e cards", o foco deveria ser:

1. **posicionamento**
2. **highlights**
3. **prova de experiência real**
4. **stack consolidada**
5. **catálogo completo como apoio**, não como protagonista

Como eu refaria a arquitetura de conteúdo

Eu não manteria o mesmo peso entre as 4 telas. Eu faria assim:

1. Home = página principal de conversão

A Home precisa vender seu perfil em 10 segundos.

Ela deveria ter, nessa ordem:

Hero forte

- nome
- título claro
- resumo de posicionamento
- CTAs principais

Exemplo de direcionamento:

- **Desenvolvedor Full Stack com foco em Front-end**
- forte em Angular, TypeScript, design systems e experiência em produtos corporativos
- botão de **Ver projetos em destaque**
- botão de **Baixar currículo**
- botão de **GitHub / LinkedIn / Contato**

Hoje a Home antiga mostra introdução pessoal, idade, estado civil, nacionalidade, telefone, e-mail, filtro de tecnologia e gráficos. Isso até tem valor, mas não deveria abrir o site desse jeito. Esse tipo de dado pesa mais pra “perfil cadastral” do que pra “portfólio memorável”. GitHub

O que eu mudaria:

- tiraria idade, estado civil e nacionalidade do topo
- telefone/e-mail podem ficar no footer ou numa seção “Contato”
- manteria a parte humana, mas em formato de **bio curta**
- colocaria logo abaixo do hero uma seção **Highlights**

2. Highlights = teu verdadeiro protagonista

Você já falou a visão certa: os principais projetos precisam ficar **bem visíveis na home**.

Eu faria uma seção com **3 a 6 projetos em destaque**, cada card contendo:

- thumbnail/screenshot
- nome
- problema resolvido
- stack principal
- seu papel
- contexto: pessoal / acadêmico / profissional
- botão de **Deploy**
- botão de **GitHub**
- botão de **Ver case**

Hoje o JSON de projetos já separa informações como ambiente, descrição, links e tecnologias, então isso já te dá uma base boa para uma camada de destaque em cima do catálogo completo. GitHub +1

Minha sugestão de highlights prováveis:

- Hans UI Design Library
- novo Portfolio
- Github Consumer
- algum projeto Angular mais sólido
- algum projeto full stack com backend separado
- 1 case profissional descrito sem expor código proprietário

3. Experiences = menos lista, mais narrativa

Na tela antiga, Experiences cruza jobs, customers, projects e technologies. Isso mostra volume, mas pode cansar. GitHub +1

Eu mudaria para formato de **timeline de carreira**, com cada experiência abrindo um bloco assim:

- empresa
- período
- cargo
- contexto do produto/cliente
- responsabilidades
- tecnologias mais relevantes
- entregas / impactos
- projetos relacionados

E deixaria clientes e tecnologias como **chips/accordions** dentro da experiência, não como grandes agrupamentos paralelos.

4. Skills = sair do "Pokédex de tech"

A tela antiga tem formações, idiomas e uma lista grande de tecnologias com filtros por nome, nível, tipo e stack. Isso é útil, mas do jeito tradicional vira uma parede de informação. GitHub +1

Eu reorganizaria em 3 camadas:

Camada 1 — core stack

- Angular
- TypeScript
- JavaScript
- HTML/CSS/SCSS
- Node
- SQL
- Git
- Design Systems

Camada 2 — experiência complementar

- React
- Laravel
- Express
- Bootstrap
- Chart.js / ECharts
- Azure / Jenkins / Vercel
- Swagger
- bancos

Camada 3 — estudando / explorando

- Docker
- GCP
- MongoDB
- Socket.io
- React Native
- Unity

Você já modelou níveis como Advanced, Intermediate, Beginner e Studing no JSON. Isso é bom, mas eu trocaria a apresentação de "nível" genérico por algo mais confiável, tipo:

- **uso frequente**
- **uso profissional**
- **já utilizei**
- **estudando atualmente**

Fica mais honesto e menos subjetivo.

Sobre os gráficos da Home e de Skills

Sendo brutalmente sincero: **gráfico em portfólio pessoal costuma mais atrapalhar do que ajudar**, a não ser que ele responda uma pergunta real.

No teu caso, eu só manteria gráfico se ele dissesse algo tipo:

- distribuição da experiência por stack
- evolução por anos
- mix de projetos por contexto (profissional/pessoal/formação)

Mesmo assim, eu usaria no máximo **1 ou 2 gráficos**, pequenos, com leitura muito rápida.

Se virar "dashboard de mim mesmo", o recrutador fecha a aba mentalmente.

O que eu melhoraria em cada tela

Home

Trocar:

- foco em dados pessoais
- filtro de tech logo no início
- gráficos cedo demais

Por:

- hero forte
- highlights
- mini resumo da carreira

- core stack
- CTA para projetos/contato

Experiences

Trocar:

- agrupamentos extensos
- muita relação cruzada exposta ao mesmo tempo

Por:

- timeline
- cards de impacto
- projetos associados por experiência
- stack usada por experiência

Skills

Trocar:

- listão bruto
- taxonomia excessivamente técnica

Por:

- categorias mais humanas
- destaque da stack principal
- filtros melhores
- badges de contexto: profissional, acadêmico, pessoal, estudando

Projects

Essa é a tela que mais pode brilhar.

Eu faria:

- busca
- filtros por stack, contexto, ano, status e destaque
- toggle de visualização: **grid** / **case study**
- cards com preview real
- projeto com página própria `/projects/:slug`

Hoje a tela antiga já tem filtros úteis por nome, environment, data e tecnologia. A base lógica está boa. O problema é mais de **produto e apresentação** do que de funcionalidade.

GitHub

Melhorias de design que deixariam o site mais moderno

Aqui é onde dá pra cozinhar bonito sem fritar UX.

Visual direction

Eu iria num visual:

- dark mode elegante por padrão
- muitos espaços em branco
- cards grandes
- bordas suaves
- blur/glass bem controlado
- tipografia forte
- ícones consistentes
- microinterações leves

Layout

- largura máxima bem controlada
- hero em 2 colunas
- grid responsivo de 12 colunas
- seção com variação de ritmo visual, pra não virar “bloco em cima de bloco”

Componentização com tua lib

Como você já tem a `hans-ui-design-lib`, eu usaria ela como base para:

- Button
- Card
- Tag/Badge
- Input
- Select
- Modal / Drawer
- Tabs
- Accordion
- Tooltip
- Empty state
- Skeleton
- Chart wrapper

Isso é ouro porque o portfólio vira também **case real da tua própria design system**. Aí você não só fala que fez uma lib — você **prova**.

Melhorias de conteúdo que aumentam muito o nível

1. Transformar projeto em case

Em vez de só “nome + stack + link”, cada highlight deveria responder:

- qual era o problema?

- qual era teu papel?
- quais decisões técnicas você tomou?
- quais desafios enfrentou?
- o que faria diferente hoje?

Isso sobe absurdamente a percepção de senioridade.

2. Separar projetos por tipo

Você falou que quer mostrar tudo. Beleza. Mas não deixa tudo com o mesmo peso.

Eu separaria:

- **Highlights**
- **Projetos profissionais**
- **Projetos pessoais**
- **Projetos acadêmicos/estudos**
- **Labs / experimentos**

3. Mostrar maturidade, não só ferramenta

Além de tech stack, crie uma seção tipo:

- Clean Code
- SOLID
- Design Systems
- Componentização
- Responsividade
- Acessibilidade
- Testes
- APIs REST
- Arquitetura Front-end
- Performance
- Integração com design tokens

No app antigo você mesmo já deixou anotado que queria incluir conceitos como paradigmas, architectures e design patterns. Isso faz bastante sentido no remake. GitHub

4. Colocar screenshots de verdade

Você já tinha anotado isso como TODO, e sim, faz muita diferença. GitHub

Card sem imagem hoje perde força.
Para highlights, eu colocaria:

- thumbnail principal
- 2 ou 3 prints internos

- talvez vídeo/GIF curto

Backend: .NET ou NestJS?

Minha visão, na lata:

Vai de NestJS se:

- o objetivo principal for **velocidade**
- você quer manter stack mais alinhada com front
- quer subir logo API de conteúdo/admin
- quer DX mais fluida no ecossistema TS

Vai de .NET se:

- você quer usar o projeto como vitrine forte de backend enterprise
- quer reforçar currículo full stack corporativo
- pensa em arquitetura mais robusta desde cedo
- quer mostrar domínio fora do ecossistema JS

Meu take pra esse caso

Para **portfólio pessoal**, eu iria de **NestJS**.

Motivo:

- menor fricção com Angular
- shared contracts/types
- menos troca de contexto mental
- mais rápido de evoluir
- te permite focar mais no que realmente vai aparecer: **UX, conteúdo, admin e cases**

Só iria de .NET se você quiser explicitamente que o portfólio sirva também como **case de backend enterprise em C#**.

Como eu estruturaria o novo projeto

Front-end

- Angular
- standalone components
- signals
- route-level lazy loading
- Tailwind + SCSS
- tua design system como base visual
- SSR/prerender se fizer sentido pro SEO

Back-end

- NestJS
- módulo de auth simples para admin
- módulo de projects
- módulo de experiences
- módulo de skills
- módulo de formations/certifications
- módulo de highlights
- módulo de contact

Banco

Pra começo:

- PostgreSQL

Admin

Mesmo que simples:

- login
- CRUD de projetos
- upload de screenshots
- tags/stack
- toggle de destaque
- ordem de exibição

Assim você para de depender de JSON hardcoded e ganha escalabilidade real.

Modelo de dados que faria mais sentido

Eu modelaria entidades assim:

- `Project`
- `ProjectLink`
- `ProjectImage`
- `Technology`
- `Experience`
- `Formation`
- `Certification`
- `Highlight`
- `Tag`

E cada projeto teria:

- slug
- title

- shortDescription
- fullCase
- context
- visibility
- featured
- technologies[]
- links[]
- images[]
- startDate/endDate
- company/client opcional

O que eu evitaria no remake

- excesso de gráficos
- excesso de filtros logo de cara
- informação pessoal demais no hero
- card pequeno com texto apertado
- taxonomia complexa demais
- mostrar tudo no mesmo nível de importância
- muito “badge por badge por badge” sem narrativa

O que eu faria pra ficar realmente memorável

Três coisas:

1. Home com personalidade

Sem ser cringe, sem ser carnaval de animação.
Algo clean, mas com identidade.

2. Cases bons

Um recrutador ou tech lead lembrar de você por:

- “o cara do design system”
- “o cara que organiza bem a experiência”
- “o cara que mostra decisão técnica, não só tecnologia”

3. Consistência visual

Se tua lib estiver sendo usada de ponta a ponta, teu portfólio vira quase uma showroom dela. Isso é fortíssimo.

Meu plano recomendado de execução

Eu faria em 4 fases:

Fase 1 — fundação

- arquitetura Angular
- tema visual
- design system integrada
- rotas e layout base

Fase 2 — conteúdo principal

- Home
- Highlights
- Projects com dados reais
- páginas individuais de projeto

Fase 3 — credibilidade

- Experiences
- Skills
- Formations
- Contact
- SEO básico

Fase 4 — refinamento

- backend/admin
- screenshots
- animações leves
- analytics
- otimização/performance

Resumo da call

Teu portfólio antigo já prova que você tem bastante material. O problema não é falta de conteúdo — é **curadoria, hierarquia e storytelling**.

O remake ideal deveria:

- colocar **highlights na cara**
- reduzir a sensação de “catálogo bruto”
- transformar projetos em **cases**
- usar tua **design system como diferencial real**
- deixar skills e experiences mais estratégicas
- usar backend para conteúdo dinâmico e escalável

No geral, teu direcionamento novo está muito melhor que o antigo: **Angular + TS + design system própria + possibilidade de backend** é uma base bem mais madura.

GitHub +2

Se você quiser, eu posso no próximo passo montar **uma arquitetura completa de páginas/seções do novo portfólio**, já com sugestão de layout e quais componentes da tua lib usar em cada parte.

Fontes

CV - Victor Hanszman (E...
PDF

CV - Victor Hanszman (PT...
PDF

Beleza, concordo com vc em vários pontos!
Sendo assim, me ajude a estabelecer um plano de implementação no https://github.com/Hanszman/hans-portfolio-app, que vai ser o remake do https://github.com/Hanszman/victor_hanszman_portfolio-old. Estabeleça esse plano de implementação para o Codex considerando tudo o que já falamos, considerando também meu currículo atualizado (estou te enviando os pdfs das versões ingles e portugues com todos os highlights e experiencias que tive onde trabalhei e estudei) então pode pegar varias informações de lá para se atualizar e também do meu linkedin: https://www.linkedin.com/in/victor-hanszman-b1362215b/ ... e considere também alguns outros pontos:
-Vamos utilizar minha lib https://github.com/Hanszman/hans-ui-design-lib via CDN nesse projeto Angular, como já iniciliazamos ela lá. Vamos utilizar todos os componentes que vc citou mesmo, mas não entendi o que seria esse componente "Accordion" que vc citou... se vc enxergar a necessidade de ter um accordion no remake do portfolio, sinta-se a vontade para propor a criação dele na lib, ai mencione isso no plano de implementação também (lembrando de seguir o padrão da lib igual dos outros componentes e igual descrevi no arquivo TODOLIST.txt (deve utilizar: clean code, dry, srp, solid, passar com 100% de coverage nos tests, deve utilizar tailwind nos scss, passar no lint e nos builds, além de funcionar perfeitamente no storybook... e deve ter os 6 arquivos padrão (tsx,scss,types,test,mdx,stories) e, quando necessário, com os helpers (e test))... e também não entendi o que seria esse "Empty state" que vc mencionou, pq não tem esse componente na lib, ele é apenas um estado que é usado em vários componentes da lib... sobre

o sistema de grid que vc comentou, será que criamos também essa lógica no desing lib? ...mas enfim! é isso sobre a lib
-Pretendo utilizar .NET justamente para esses motivos que vc falou (usar o projeto como vitrine forte de backend enterprise, reforçar currículo full stack corporativo, pensando em arquitetura mais robusta desde cedo e mostrar domínio fora do ecossistema JS)... até porque já pretendo usar nest/node em outros projetos que pretendo fazer posteriormente e queria diversificar tendo pelo menos um projeto com C#/.NET no backend, sacou?
-Pretendo também fazer dois novos projetos além desse remake (não precisa preocupar em definir nada deles agora, mas é só pra ter como espaço para citação futura nos highlights do portfolio): o https://github.com/Hanszman/hans-game-tracker-app que vai ser como se fosse um backloggd/ggapp/trello/howlongtobeat de games e pretendo também fazer um site de exposição de produtos da loja da minha esposa e da minha sogra (elas tem juntas uma empresa de biscoitos e artesanatos)... e pra esses dois sites pretendo utilizar next/react/ts no front e nest/node/ts no back... pretendo fazer eles na sequencia do remake do portfolio e ja incluir eles la nos highlights... e em ambos vou utilizar a lib de design que desenvolvi
-Inclua React como uma tecnologia core stack por favor, pois apesar de eu não ter trabalhado profissionalmente com ele, eu utilizo em todos os meus projetos pessoais... sei q vc só deu um exemplo, mas procure destacar as tecnologias que eu destaquei no currículo como sendo as prioritárias na hora de classificar (estou falando das camadas que vc citou)
-sobre a termologia/nomenclatura dos "níveis" das tecnologias que vc citou, não concordo muito... acho que frequencia de uso (frequente/já utilizei) e aspecto de uso (profissional/estudo) e nível (avançado/intermediario/básico) são 3 coisas diferentes e poderiamos classificar cada uma das tecnologias com esses 3 tipos de coisa, o que acha?
-sobre os gráficos, eu gostaria que tivesse uma tela meio dashboard sim relacionando utilização, tempo, projetos, tecnologias, etc, com vários gráficos mostrando informações diferentes, até pra mostrar que tenho

experiencia em criação de dashboards analíticos... concordo com vc que não deveria estar na página inicial... mas acho que seria interessante ter uma tela pra isso
-sobre as imagens dos ícones de tecnologias, minha foto, dentre outras coisas, pode copiar direto do projeto do portfolio antigo mesmo...
-Divida em etapas de implementação:
* Primeiro vamos reformular as telas, criando-as já no repo novo e já na estrutura de layout novo que vc propos e já utilizando também a lib de design... porém vamos fazer isso tudo sem backend ainda, utilizando os mesmos jsons do projeto anterior (com as entidades: experiences, projects e skills)... o projeto também vai ter mecanica de tradução com o i18n (com os idiomas pt-br, en-us e depois mais pra frente vamos expandir também para espanhol e frances)... OBS: inclusive, devemos fazer algum componente padrão relacionado a seleção de linguagem (dropdown) no design lib para reutilizar nos meus outros projetos? fica ai questionamento se faz sentido (ai no design teriam listados ja os idiomas e em cada projeto que usaria ele teriam os jsons com os textos de tradução)... pode ter também um dark-mode no remake do portfolio (faremos isso criando dois hansthemes diferentes, um com cores mais claras de fundo e coloridas de interação e outro com cores mais escuras de fundo, e alternaremos entre essas combinations com um toggle que será um componente do novo projeto/remake (e vai usar o toggle do design lib) e vamos passar por parametro esse obj/combinação de cores pelo sistema de cores que ja temos estabelecido na lib.... Além de tudo isso, podemos criar componentes específicos do remake no proprio projeto/repo do remake mesmo, como por exemplo seu navbar/menu (que usará bastante o dropdown da lib), esse toggle personalizado de dark-mode (que também usará a lib por baixo dos panos) dentre outras coisas... Como essa primeira etapa ficou grandona, se quiser subdividir em etapar menores dentro dela também pode...
* Depois vamos fazer a pagina de admin, como vc comentou, mas em uma etapa posterior com CRUDs para cadastrar os objetos posteriormente (usando postgresql no neon db mesmo pq é free) e criando um backend/api em .NET/C#... mas ai sobre as entidades, eu concordo com a maioria que vc colocou, mas da pra colocar também

languages (os idiomas que eu falo mesmo) e acho que links e imagens podem ser tabelas separadas (que podem ter relação N:N pra project, experience, formation, etc... ai pode ter a descrição de cada um também (por exemplo no link, se é do git, do npm, do deploy, etc))... sobre o highlight acho que não deveria ser uma tabela e sim uma coluna em cada project, experience, formation, etc (boolean mesmo, com true/false, se é para aparecer como destaque ou não)... pode ter a tabela de customers e jobs também (fazem parte do experiences mas são coisas diferentes como vc pode ver)... e como vc pode ver também nos jsons, varias tecnologias, projects, experiencies, etc se relacionam entre si, então tudo isso seria contemplado na estruturação do banco de dados

Enfim! Faça o plano por favor em formato .md

Pensou por 6m 10s >

Fechei o plano em `.md` pra você passar pro Codex:

Baixar plano-implementacao-hans-portfolio-app.md

Montei ele considerando:

- o repo novo em Angular já iniciado com Tailwind, SCSS, testes e routing standalone, além de module federation já instalado. GitHub +1
- os TODOs e a estrutura do portfólio antigo, que já apontavam pontos como screenshots, modais, links entre entidades, paginação, responsividade, gráficos e reorganização da tela de skills. GitHub +1
- tua design lib com suporte a React via npm e Angular via CDN/web components, além do sistema de temas dinâmicos. GitHub
- teu currículo atualizado, com 7+ anos, foco em Angular/TypeScript/React, design system, micro front-end, modernização de legado, dashboards, SOLID, TDD, CI/CD, Docker e backend complementar em Node/PHP/C#. CV - Victor Hanszman (EN) CV - Victor Hanszman (PT-BR)

Também alinhei no plano os pontos que você cravou:

- React entrou como **core stack**;
- classificação de tecnologias ficou em **3 eixos separados**: frequência, contexto e nível;
- dashboard ganhou **tela própria**, em vez de poluir a home;

- backend ficou com **.NET + PostgreSQL/Neon**;
- `highlight` virou **boolean nas entidades**, não tabela;
- `links`, `images`, `languages`, `customers` e `jobs` entraram na modelagem futura;
- `Accordion` foi proposto como novo componente da lib;
- `Empty state` ficou tratado como **padrão visual**, não componente obrigatório da lib;
- grid ficou recomendado como **layout primitive local primeiro**, sem extrair cedo demais pra lib.

Se você quiser, no próximo passo eu posso te entregar uma **versão 2 desse mesmo plano já em formato de prompt operacional pro Codex**, tipo bem mastigado em checklist de execução por sprint/PR.

Fontes

Fontes

Dei uma olhada no seu plano de implementação que vc estruturou no md e gostei bastante... só tenho algumas observações:
-Deixe claro que estamos usando o angular na versão 20.3.6, ou seja, quero que sejam usadas as novas abordagens do framework, utilizando signals para gerenciamento de estado (com computed, effect, etc), standalone components (ao invés de usar modules), nova sintaxe para condicionais e loops no html (com @if e @for por exemplo ao invés de ng-if e ng-for), além das novas formas de declarar input() e output() do Angular, como também chamar serviços via inject(), etc... deixe tudo modernizado por favor
-Além disso, não entendi algumas coisas na estrutura de pastas que vc propos do frontend... tente fazer algo que se assemelhe melhor ao front antigo (com componentes dividos entre pastas de layout e pages por exemplo)... não entendi o que seria o app-shell por exemplo, o nome de page-header ficou legal mas o footer então não deveria ser page-footer? o que seria o section-wrapper, seria o container da pagina? não sei se entendi... o utils e o helpers não seriam a mesma coisa? e o shared não deveria estar dentro de core? pq entendo que tanto o core como o shared são coisas comuns a configurações de toda aplicação, mas acho q o shared (por ser mais específico de

components, diretivas, etc reutilizaveis) poderia estar dentro ali do core também... ai tira esse utils e deixa só helpers mesmo, acho que ambos devem ocupar a mesmo papel... e a pasta do i18n poderia chamar translation mesmo (pq vai que mudamos de lib um dia, o nome da pasta, funcionalidade de tradução não deveria estar diretamente atrelado à lib que usaremos, para não ficar acoplado demais... a não ser q esse seja o padrão de nome de pasta quando se usa essa lib, ai poderia manter) e e essa pasta deveria estar dentro de core também, pelo oq eu entendo... então sendo assim teriam 3 pastas "pai" dentro de src/app => core, layout e pages (ou features, ai dentro de features pode ter a pasta do que seria pages e outras cosias que não seriam pages em outra pasta), alem dos arquivos raiz de app e routes (que inclusive, se houverem muitas rotas filhas, pode ter uma pasta organizada de rotas também)... enfim, organize melhor o sistema do front, ficando mais proximo disso que comentei
-To refletindo sobre o sistema e to achando melhor fazermos o backend primeiro, para não termos retrabalho na hora de implementar a integração entre ambos... ai na etapa do backend, que agora será a primeira, pode fazer todos os CRUDS (incluindo de todas as entidades, pode incluir também o login e o sistema de admin (onde haverão os cadastros e parte onde somente eu irei ter acesso, ja q o usuário normal (que não precisa logar) terá acesso somente aos gets ne, ou seja, a navegação normal do meu portfolio))... pode incluir isso tudo e também os scripts de banco de dados (incluindo a criação das tabela e dados (ai pode ser basear nos jsons do portfolio anterior para fazer os inserts dos dados das entidades, relacionamentos, etc)... e pode incluir também um ORM para facilitar a integração com o banco: pode ser o mais usado e comum do C#/.NET mesmo, acho q é o entity framework certo? não sei, mas pode escolher a melhor opção pra esse cenário... e coloque documentação dos endpoints também com swagger, etc... e crie um readme, parecido com esses que criei pros meus outros projetos novos (hans-ui-design-lib e hans-portfolio-app) por exemplo, contendo todo o passo a passo para usar a api, todos os scripts e comandos necessários para clonar, rodar o projeto, etc... alem de todo o histórico de scripts

rodados para implementar o projeto, como libs que foram instaladas (se houverem, pois não sei exatamente como funciona no .NET se é parecido com o npm ou não... inclusive deixe o projeto do backend bem simples de entender, pois ainda sou iniciante em .NET e C#)
-Enfim! faça todas essas alterações no plano de implementação md e mantenha todo o resto do jeito que vc estimou ai por favor!

Reflexão interrompida >

Dei uma olhada no seu plano de implementação que vc estruturou no md e gostei bastante... só tenho algumas observações: -Deixe claro que estamos usando o angular na versão 20.3.6, ou seja, quero que sejam usadas as novas abordagens do framework, utilizando signals para gerenciamento de estado (com computed, effect, etc), standalone components (ao invés de usar modules), nova sintaxe para condicionais e loops no html (com @if e @for por exemplo ao invés de ng-if e ng-for), além das novas formas de declarar input() e output() do Angular, como também chamar serviços via inject(), etc... deixe tudo modernizado por favor -Além disso, não entendi algumas coisas na estrutura de pastas que vc propos do frontend... tente fazer algo que se assemelhe melhor ao front antigo (com componentes dividos entre pastas de layout e pages por exemplo)... não entendi o que seria o app-shell por exemplo, o nome de page-header ficou legal mas o footer então não deveria ser page-footer? o que seria o section-wrapper, seria o container da pagina? não sei se entendi... o utils e o helpers não seriam a mesma coisa? e o shared não deveria estar dentro de core? pq entendo que tanto o core como o shared são coisas comuns a configurações de toda aplicação, mas acho q o shared (por ser mais específico de components, diretivas, etc reutilizaveis) poderia estar dentro ali do core também... ai tira esse utils e deixa só helpers mesmo, acho que ambos devem ocupar a mesmo papel... e a pasta do i18n poderia chamar translation mesmo (pq vai que mudamos de lib um dia, o nome da pasta, funcionalidade de tradução não deveria

estar diretamente atrelado à lib que usaremos, para não ficar acoplado demais... a não ser q esse seja o padrão de nome de pasta quando se usa essa lib, ai poderia manter) e e essa pasta deveria estar dentro de core também, pelo oq eu entendo... então sendo assim teriam 3 pastas "pai" dentro de src/app => core, layout e pages (ou features, ai dentro de features pode ter a pasta do que seria pages e outras cosias que não seriam pages em outra pasta), alem dos arquivos raiz de app e routes (que inclusive, se houverem muitas rotas filhas, pode ter uma pasta organizada de rotas também)... enfim, organize melhor o sistema do front, ficando mais proximo disso que comentei -To refletindo sobre o sistema e to achando melhor fazermos o backend primeiro, para não termos retrabalho na hora de implementar a integração entre ambos... ai na etapa do backend, que agora será a primeira, pode fazer todos os CRUDS (incluindo de todas as entidades, pode incluir também o login e o sistema de admin (onde haverão os cadastros e parte onde somente eu irei ter acesso, ja q o usuário normal (que não precisa logar) terá acesso somente aos gets ne, ou seja, a navegação normal do meu portfolio))... pode incluir isso tudo e também os scripts de banco de dados (incluindo a criação das tabela e dados (ai pode ser basear nos jsons do portfolio anterior para fazer os inserts dos dados das entidades, relacionamentos, etc)... e pode incluir também um ORM para facilitar a integração com o banco: pode ser o mais usado e comum do C#/.NET mesmo, acho q é o entity framework certo? não sei, mas pode escolher a melhor opção pra esse cenário... e coloque documentação dos endpoints também com swagger, etc... e crie um readme, parecido com esses que criei pros meus outros projetos novos (hans-ui-design-lib e hans-portfolio-app) por exemplo, contendo todo o passo a passo para usar a api, todos os scripts e comandos necessários para clonar, rodar o projeto, etc... alem de todo o histórico de scripts rodados para implementar o projeto, como libs que foram instaladas (se houverem, pois não sei exatamente como funciona no .NET se é parecido com o npm ou não... inclusive deixe o projeto do backend bem simples de entender, pois ainda sou iniciante em .NET e C#) -Enfim! faça todas essas alterações no plano de implementação md e mantenha todo o resto do jeito que vc estimou ai

por favor! Ai relembrando: inicialmente faremos backend completo (incluindo banco de dados), depois frontend completo (remake do portfolio atual), depois integração entre eles

Reflexão interrompida >

Dei uma olhada no seu plano de implementação que vc estruturou no md e gostei bastante... só tenho algumas observações:
-Deixe claro que estamos usando o angular na versão 20.3.6, ou seja, quero que sejam usadas as novas abordagens do framework, utilizando signals para gerenciamento de estado (com computed, effect, etc), standalone components (ao invés de usar modules), nova sintaxe para condicionais e loops no html (com @if e @for por exemplo ao invés de ng-if e ng-for), além das novas formas de declarar input() e output() do Angular, como também chamar serviços via inject(), etc... deixe tudo modernizado por favor
-Além disso, não entendi algumas coisas na estrutura de pastas que vc propos do frontend... tente fazer algo que se assemelhe melhor ao front antigo (com componentes dividos entre pastas de layout e pages por exemplo)... não entendi o que seria o app-shell por exemplo, o nome de page-header ficou legal mas o footer então não deveria ser page-footer? o que seria o section-wrapper, seria o container da pagina? não sei se entendi... o utils e o helpers não seriam a mesma coisa? e o shared não deveria estar dentro de core? pq entendo que tanto o core como o shared são coisas comuns a configurações de toda aplicação, mas acho q o shared (por ser mais específico de components, diretivas, etc reutilizaveis) poderia estar dentro ali do core também... ai tira esse utils e deixa só helpers mesmo, acho que ambos devem ocupar a mesmo papel... e a pasta do i18n poderia chamar translation mesmo (pq vai que mudamos de lib um dia, o nome da pasta, funcionalidade de tradução não deveria estar diretamente atrelado à lib que usaremos, para não ficar acoplado demais... a não ser q esse seja o padrão de nome

de pasta quando se usa essa lib, ai poderia manter) e e essa pasta deveria estar dentro de core também, pelo oq eu entendo... então sendo assim teriam 3 pastas "pai" dentro de src/app => core, layout e pages (ou features, ai dentro de features pode ter a pasta do que seria pages e outras cosias que não seriam pages em outra pasta), alem dos arquivos raiz de app e routes (que inclusive, se houverem muitas rotas filhas, pode ter uma pasta organizada de rotas também)... enfim, organize melhor o sistema do front, ficando mais proximo disso que comentei
-To refletindo sobre o sistema e to achando melhor fazermos o backend primeiro, para não termos retrabalho na hora de implementar a integração entre ambos... ai na etapa do backend, que agora será a primeira, pode fazer todos os CRUDS (incluindo de todas as entidades, pode incluir também o login e o sistema de admin (onde haverão os cadastros e parte onde somente eu irei ter acesso, ja q o usuário normal (que não precisa logar) terá acesso somente aos gets ne, ou seja, a navegação normal do meu portfolio))... pode incluir isso tudo e também os scripts de banco de dados (incluindo a criação das tabela e dados (ai pode ser basear nos jsons do portfolio anterior para fazer os inserts dos dados das entidades, relacionamentos, etc)... e pode incluir também um ORM para facilitar a integração com o banco: pode ser o mais usado e comum do C#/.NET mesmo, acho q é o entity framework certo? não sei, mas pode escolher a melhor opção pra esse cenário... e coloque documentação dos endpoints também com swagger, etc... e crie um readme, parecido com esses que criei pros meus outros projetos novos (hans-ui-design-lib e hans-portfolio-app) por exemplo, contendo todo o passo a passo para usar a api, todos os scripts e comandos necessários para clonar, rodar o projeto, etc... alem de todo o histórico de scripts rodados para implementar o projeto, como libs que foram instaladas (se houverem, pois não sei exatamente como funciona no .NET se é parecido com o npm ou não... inclusive deixe o projeto do backend bem simples de entender, pois ainda sou iniciante em .NET e C#)
-Enfim! faça todas essas alterações no plano de implementação md e mantenha todo o resto do jeito que vc estimou ai por favor! Ai relembrando: inicialmente

faremos backend completo (incluindo banco de dados), depois frontend completo (remake do portfolio atual), depois integração entre eles

Ótimas observações! Vou ajustar tudo no plano, com Angular 20.3.6 e as abordagens mais recentes (signals, standalone, nova sintaxe e inject). Vou também reorganizar a estrutura de pastas, deixando mais alinhada ao modelo anterior: core, layout, e pages/features, unificando helpers, renomeando i18n para translation dentro de core, e renomeando footer, etc., de forma mais clara.

Dei uma olhada no seu plano de implementação que vc estruturou no md e gostei bastante... só tenho algumas observações:
-Deixe claro que estamos usando o angular na versão 20.3.6, ou seja, quero que sejam usadas as novas abordagens do framework, utilizando signals para gerenciamento de estado (com computed, effect, etc), standalone components (ao invés de usar modules), nova sintaxe para condicionais e loops no html (com @if e @for por exemplo ao invés de ng-if e ng-for), além das novas formas de declarar input() e output() do Angular, como também chamar serviços via inject(), etc... deixe tudo modernizado por favor
-Além disso, não entendi algumas coisas na estrutura de pastas que vc propos do frontend... tente fazer algo que se assemelhe melhor ao front antigo (com componentes dividos entre pastas de layout e pages por exemplo)... não entendi o que seria o app-shell por exemplo, o nome de page-header ficou legal mas o footer então não deveria ser page-footer? o que seria o section-wrapper, seria o container da pagina? não sei se entendi... o utils e o helpers não seriam a mesma coisa? e o shared não deveria estar dentro de core? pq entendo que tanto o core como o shared são coisas comuns a configurações de toda aplicação, mas acho q o shared (por ser mais específico de components, diretivas, etc reutilizaveis) poderia estar dentro ali do core também... ai tira esse utils e deixa só

helpers mesmo, acho que ambos devem ocupar a mesmo papel... e a pasta do i18n poderia chamar translation mesmo (pq vai que mudamos de lib um dia, o nome da pasta, funcionalidade de tradução não deveria estar diretamente atrelado à lib que usaremos, para não ficar acoplado demais... a não ser q esse seja o padrão de nome de pasta quando se usa essa lib, ai poderia manter) e e essa pasta deveria estar dentro de core também, pelo oq eu entendo... então sendo assim teriam 3 pastas "pai" dentro de src/app => core, layout e pages (ou features, ai dentro de features pode ter a pasta do que seria pages e outras cosias que não seriam pages em outra pasta), alem dos arquivos raiz de app e routes (que inclusive, se houverem muitas rotas filhas, pode ter uma pasta organizada de rotas também)... enfim, organize melhor o sistema do front, ficando mais proximo disso que comentei
-To refletindo sobre o sistema e to achando melhor fazermos o backend primeiro, para não termos retrabalho na hora de implementar a integração entre ambos... ai na etapa do backend, que agora será a primeira, pode fazer todos os CRUDS (incluindo de todas as entidades, pode incluir também o login e o sistema de admin (onde haverão os cadastros e parte onde somente eu irei ter acesso, ja q o usuário normal (que não precisa logar) terá acesso somente aos gets ne, ou seja, a navegação normal do meu portfolio))... pode incluir isso tudo e também os scripts de banco de dados (incluindo a criação das tabela e dados (ai pode ser basear nos jsons do portfolio anterior para fazer os inserts dos dados das entidades, relacionamentos, etc)... e pode incluir também um ORM para facilitar a integração com o banco: pode ser o mais usado e comum do C#/.NET mesmo, acho q é o entity framework certo? não sei, mas pode escolher a melhor opção pra esse cenário... e coloque documentação dos endpoints também com swagger, etc... e crie um readme, parecido com esses que criei pros meus outros projetos novos (hans-ui-design-lib e hans-portfolio-app) por exemplo, contendo todo o passo a passo para usar a api, todos os scripts e comandos necessários para clonar, rodar o projeto, etc... alem de todo o histórico de scripts rodados para implementar o projeto, como libs que foram instaladas (se houverem, pois não sei exatamente como

funciona no .NET se é parecido com o npm ou não... inclusive deixe o projeto do backend bem simples de entender, pois ainda sou iniciante em .NET e C#)
-Enfim! faça todas essas alterações no plano de implementação md e mantenha todo o resto do jeito que vc estimou ai por favor! Ai relembrando: inicialmente faremos backend completo (incluindo banco de dados), depois frontend completo (remake do portfolio atual), depois integração entre eles

Pensou por 4m 40s >

Perfeito. Atualizei o plano com essas mudanças e deixei a ordem exatamente como você definiu:

1. **backend completo + banco**
2. **frontend completo**
3. **integração entre ambos**

Também ajustei os pontos que você pediu:

- Angular **20.3.6** explicitamente, com `signals`, `computed`, `effect`, standalone, `@if`, `@for`, `input()`, `output()`, `inject()` e abordagem moderna; a documentação oficial do Angular confirma a política de versões e recomenda ficar sempre no patch mais recente da major/minor usada. Angular +1
- backend reposicionado como **primeira etapa**
- recomendação explícita de **ASP.NET Core Web API + Entity Framework Core + PostgreSQL + Swagger/OpenAPI**, alinhada com a documentação oficial da Microsoft para APIs com banco e documentação Swagger. Microsoft Learn +2
- estrutura do front reorganizada para ficar mais próxima do que você queria: `core`, `layout`, `pages`, com `translation` dentro de `core`, `shared` dentro de `core`, `helpers` no lugar de `utils`, `page-footer` pareado com `page-header`, e explicação clara de `public-shell` / `admin-shell`
- README da API agora entrou como parte obrigatória da etapa de backend
- mantive React como **core stack**
- mantive dashboard em tela própria
- mantive `highlight` como boolean nas entidades
- mantive `links`, `images`, `languages`, `customers` e `jobs` na modelagem
- considerei os highlights do currículo enviado e o resumo público do LinkedIn, que está consistente com eles. CV - Victor Hanszman (EN) CV - Victor Hanszman (EN) CV - Victor Hanszman (PT-BR) LinkedIn

Aqui está a versão revisada:

Baixar plano-implementacao-hans-portfolio-app-v2.md

No próximo passo, eu transformo esse `.md` em um **prompt operacional pro Codex**, já quebrado em tasks/PRs.

Fontes

Fontes